No.	Systems Affected	Issue	Risk Rating	Impact	Recommendation
1	Test Management Tool	***Stored Cross-Site Scripting (XSS) in Inventory Module*** *It was observed that the test management tool's inventory module accepts unsanitized user input in multiple fields, including machineName, machinePurpose, machineHw, and machineNotes. A POST request to /lib/inventory/setInventory.php can be crafted to include malicious payloads such as <img src=x onerror=alert('XSS')>. These payloads are stored in the backend and executed in the browser context when any user views the inventory page, leading to a stored cross-site scripting (XSS) vulnerability. This behavior indicates improper input validation and missing output encoding on affected fields.*	**High**	1. Session hijacking, credential theft, or redirection to malicious sites 2. Affects all users who view the compromised inventory entry	1. Implement Output Encoding: Apply context-aware encoding (e.g., htmlspecialchars() in PHP) to all user-supplied data rendered in HTML pages. 2. Validate Input Strictly: Restrict inventory input fields
2	Test Management Tool	***Time-Based SQL Injection in Search Function*** *It was observed that the edited_by parameter in the search functionality (/lib/search/search.php) is vulnerable to time-based blind SQL injection. An attacker can inject SQL payloads such as:* *' + (SELECT * FROM (SELECT SLEEP(5))a) + '* *This leads to measurable response delays, confirming backend SQL query manipulation. No proper sanitization or parameterization appears to be in place on this input.*	**High**	1. Data Exposure: Sensitive data may be accessed, including user accounts, project details, and internal configuration. 2. System Resource Abuse: Using heavy time-based payloads (e.g., SLEEP(10)) can degrade application performance or cause denial of service.	Enforce strict server-side validation and whitelist expected values for input parameters
3	Test Management Tool	***User Role Manipulation via DevTools*** *It was observed that when an admin role is assigned to a user, it can be changed to another role by using browser developer tools. When the user visits the user management page, the role field is disabled. However, by deleting the disabled attribute using the developer tools, the role can be modified, including downgrading from admin to another role.*	**High**	Allows unauthorized modification of user roles, potentially downgrading users from an admin role to a less privileged role.	Backend must validate that the machine owner value corresponds to the authenticated session user and not accept external input for ownership.
4	Test Management Tool	***Exposed Backup Installation Path*** *It was observed that the file /install.bak/installDbInput.php was publicly accessible. This file is part of the installation process, likely to be used for database setup.*	**High**	May allow unauthorized reinstallation or exposure of DB configuration flow, leading to takeover or data leaks.	Delete or restrict access to old installer files or backup folders.
5	Test Management Tool	***Exposed Diagnostic Endpoint*** *It was observed that the CGI script /cgi-bin/printenv.pl was publicly accessible. This script typically prints all server environment variables (e.g., PATH, USER, SERVER_SOFTWARE).*	**High**	script exposes sensitive server environment details, which can help attackers gather information for further attacks such as server fingerprinting.	Disable unused CGI scripts. Remove default or test scripts from production servers.
6	Test Management Tool	***IDOR on machineOwner*** *It was observed that when submitting the form to /lib/inventory/setInventory.php, the machineOwner field is set using a numeric user ID passed in the POST body. Although the UI binds this value to the logged-in user (dropdown reflects session user), changing the ID in the request (e.g., 2 to 9) results in creating inventory owned by a different user. from POC shows user innocent.mosha has created while its user baraka.range has logged in*	**High**	This allows a logged-in user to impersonate or assign assets under other user accounts, violating authorization rules.	Delete or restrict access to old installer files or backup folders.
7	Test Management Tool	***Cross-Site Request Forgery (CSRF)*** *It was observed that the file /install.bak/installDbInput.php was publicly accessible. This file is part of the installation process, likely to be used for database setup.*	**High**	An attacker can cause state-changing actions on behalf of authenticated users, including privilege escalation, inventory tampering, and unauthorized updates.	Apply CSRF tokens on all state-changing forms and APIs, enforce SameSite=Strict on cookies, and validate
8	Test Management Tool	***Exposed Configuration Files*** *It was observed that sensitive files such as config.inc.php (configuration file), info.php (possible information disclosure script), and Dockerfile (build configuration) were directly accessible from the web server.*	**Medium**	files may expose database credentials, internal paths, PHP config details, and environment variables, which can be used in further attacks	Apply proper access controls (e.g., .htaccess, Nginx rules). Do not expose config files to the public.
9	Test Management Tool	***Directory Listing Enabled*** *It was observed that directories such as /lib/, /docs/, and /logs/ allowed full directory indexing and browsing.*	**Medium**	Attackers can discover sensitive files, unused components, or outdated libraries, and download or analyze them.	Disable directory listing via web server configuration (e.g., Options -Indexes in Apache).

10	Test Management Tool	***Exposed Development/Info Files*** *It was observed that internal documentation and development artifacts such as README.md (documentation file), CHANGELOG (version log), and contents under /vendor/ (dependency-related files) were publicly accessible.*	**Medium**	files can reveal application versions, components in use, plugin info, or other internal logic helpful to an attacker.	Remove unnecessary documentation and internal files from the public directory.
11	Test Management Tool	***Unhandled Debug Error & Path Disclosure*** *It was observed that the application displays raw debug output when a database error occurs. The stack trace includes absolute file paths such as C:\xampp\htdocs\tl-uat\lib\functions\logger.class.php, along with internal class and function names.*	**Medium**	Revealing full stack traces and filesystem paths can aid attackers in crafting targeted exploits (e.g., LFI, RCE, or further injection points), falling under CWE-200 (Information Exposure).	Disable debug mode in production environments. Log such errors server-side only. Ensure generic error messages are displayed to the user.
12	Test Management Tool	***Missing Secure, HttpOnly, and SameSite Flags on Cookies*** *It was observed that the application sets cookies without Secure, HttpOnly, or SameSite attributes. These are essential for protecting cookies from interception and misuse.*	**Medium**	Lack of these attributes may lead to cookie theft via XSS (without HttpOnly), interception over HTTP (without Secure), or CSRF (without SameSite).	Set HttpOnly to prevent JavaScript access, Secure to allow cookies only over HTTPS, and SameSite=Strict or